

KitchenBuddy

A kitchen assistant that runs on a laptop

Project report, Generative Artificial Intelligence course

ISEN Méditerranée, engineering cycle

September 2026

A team of four: AI Researcher, Data Engineer, ML Engineer, DevOps Engineer

App name: Happy Bite · Code: github.com/Infernoflamex/KitchenBuddy

Abstract

KitchenBuddy (app name: Happy Bite) is a kitchen assistant we built in six days for the Generative AI course. It keeps track of what is in the kitchen and when it goes off, reads supermarket receipts, suggests dinner from what is already there, writes recipes, and talks you through cooking one step at a time, hands-free.

The course asked for one thing that shaped every decision: the generative model had to run on our own hardware, not behind someone else's API. Our demo machine is a MacBook Air with 16 GB of memory, shared by the language model, the speech models and everything else. On that machine a model writes about 15 tokens per second, has a small working context, and makes mistakes that a cloud model would not.

We compared the four adaptation strategies seen in the course: prompting, parameter-efficient fine-tuning, full fine-tuning and retrieval-augmented generation (RAG). We chose **structured prompting, RAG, and strict checking of everything the model returns**. The knowledge the assistant needs is a household's stock, which changes every day, and a corpus of 2.1 million recipes. Neither belongs in model weights. The final model is **Gemma 4 E2B** in 4-bit, served locally through LM Studio. Retrieval uses a SQLite full-text index. A LoRA fine-tuning pipeline is written and self-tested, but we kept the training run as a second stage and explain why with numbers.

This report explains what we built, why we chose each component, how the pieces fit together, what we measured, what still does not work, and what we learned.

1. Why we built this

1.1 The problem

It is seven in the evening. You open the fridge and see half a courgette, some goat's cheese, rice and two chicken thighs that should be cooked tomorrow at the latest. You are tired. Recipe apps give you a book to search. Shopping apps give you a list. Neither knows what is on your shelves, and neither helps while your hands are covered in flour.

We wanted something closer to a friend who cooks: it knows what you have, it suggests something reasonable, and it walks you through it out loud.

1.2 The constraint that shaped everything

The course required the model to run on our own machines using the Hugging Face ecosystem. That sounds like a detail. It turned out to be the whole project.

A cloud model hides most of the engineering: huge context, fast answers, reliable JSON, tool calling that just works. A small model on a laptop behaves very differently. It reads a prompt of a few thousand tokens before it writes a single word. It writes about 15 tokens per second. It sometimes invents an ingredient, or asks for a quantity nobody could eat. So a large part of our work was not choosing a model. It was making a small model dependable enough to use while standing at the stove.

1.3 What we set out to deliver

- Track the household's stock: quantity, purchase date, storage place, and an estimate of when each item goes off.
- Fill that stock from a photographed receipt, or by voice ("I bought 300 grams of chicken").
- Suggest and write recipes from what is actually in the kitchen and the time available.
- Guide the cook step by step, mainly by voice, in English or French.
- Justify each technical choice with numbers against at least one real alternative, as the course asks.

1.4 What we actually shipped

Everything above works in the version we demo, on one MacBook Air, with no internet except for the wake word. Some parts are rougher than we would like, and section 10 lists them honestly. The fine-tuning run is the one objective we deliberately postponed.

2. How we worked

2.1 Roles

The course suggests five roles. We were four, with six days, so we merged Integration and Scrum Master into a single DevOps role. That kept coordination light. The cost was that nobody's main job was tracking progress.

Role	What it meant in practice
AI Researcher	Compared adaptation strategies and models, did the LoRA arithmetic, chose sampling settings, tested candidate models on the Mac.
Data Engineer	Chose and compared datasets, exported the 2.6 GB recipe corpus, grew the ingredient catalogue to 3,052 entries, built the Open Food Facts product index and an embedding prototype in ChromaDB.
ML Engineer	Built retrieval, the recipe checks and the "template first" path, wrote the fine-tuning scripts and the diagnostic tools.
DevOps Engineer	FastAPI backend, React frontend, local model server setup, voice integration, the macOS and Windows guides, and the day-to-day plan.

Table 1. Team roles.

2.2 Six days

Day	What we did	In the Git history
11 Sept	Framed the need, picked datasets, designed the stock and recipe formats, first FastAPI and React skeleton, first prompts.	Initial commit, "Phase 1"
12–14 Sept	Local voice: speech-to-text, text-to-speech, wake word. One API for LM Studio, Ollama or a cloud model.	"Phase 2 - Voice"
14–15 Sept	Cooking mode, the recipe index, template-first recipes, receipt reading, recipe checks.	"Phase 3 - Redo", "Phase 3 - Part 2"
15 Sept	Catalogue builder, corpus importer, Open Food Facts importer, fine-tuning scripts.	"Data related scripts"
16–17 Sept	Fixes, guides for macOS and Windows, testing on the demo Mac, receipt camera, report and demo.	"Phase 1-2-3 Correction...", "Phase 4"

Table 2. Our plan, matched to the commit history.

2.3 What went wrong along the way

Most of our lost hours had nothing to do with AI.

- **One repository on one person's account.** The code started on a member's GitHub account from an earlier competition. We had to clone it, push it to a second repository, and learn the hard way to check which remote we were pushing to.
- **A folder that existed twice.** An archive was extracted twice, which produced duplicated folders and a nested shared/shared directory. Imports in the frontend broke in ways that looked like code bugs. The fix was to flatten the tree. The lesson was to check that a file really exists before blaming the code that imports it.
- **Mac and Windows in the same team.** The Mac runs MLX models on its GPU; Windows runs GGUF models through llama.cpp. The voice stack differs too. This forced us to make every model a configuration setting instead of a dependency, and to write two installation guides.
- **"The model is broken" when it was not.** Several times the model looked broken and the real cause was configuration. It had been loaded with a 4,096-token window while we sent it 34,000 tokens. A reasoning model had its thinking mode on and spent 26 minutes before writing one recipe. Sampling settings tuned for one model stayed behind when we switched to another.
- **Python versions.** Some voice and image libraries publish nothing for Python 3.13 or 3.14, so we pinned 3.11 and 3.12.

We came out of this with four habits: measure before diagnosing; write next to each setting which model it was tuned for; treat model output as untrusted input; and when a deterministic check can do the job, prefer it to hoping the model gets better.

3. The big decision: prompting, RAG or fine-tuning?

3.1 What we had to work with

- **Data that changes every day and is different for every household.** The stock in a fridge cannot live in a model's weights.
- **A lot of existing knowledge.** Millions of recipes already exist, written by people who cooked them. We need to find the right one and follow it, not memorise a blurry average of all of them.
- **One laptop.** 16 GB of shared memory for inference, and no dedicated GPU for training.
- **A person waiting at the stove.** A spoken answer has to start within a couple of seconds.

3.2 Comparing the four options

Strategy	Fresh data?	Cost to update	Training data needed	Doable on our laptop?	What it would fix for us
Prompting only	Yes, in context	None	None	Yes	Tone, rules, output format
RAG	Yes, re-index	Minutes	None, just a corpus	Yes	Knowledge, sources, grounding
LoRA / QLoRA	No, frozen	1–2 h per run on one GPU	Hundreds to thousands of examples	On NVIDIA yes; on our Mac only a 1.5B model	Format, ingredient ids, style
Full fine-tuning	No, frozen	Days, several GPUs	Tens of thousands	No	Same as LoRA, far more expensive

Table 3. The four strategies against our constraints.

The deciding argument was freshness. Section 11.3 of the course puts it simply: use retrieval for knowledge that changes or must be cited, and fine-tuning for behaviour. A fridge changes every day. A recipe should come with its source. Both point to retrieval. Fine-tuning a model on the contents of a fridge would make no sense; putting the fridge in the prompt costs nothing.

3.3 Doing the maths on LoRA

We still wanted to know what fine-tuning would cost, using the method of course section 9.3. LoRA freezes the original weights and trains two small matrices per adapted layer. For a matrix of size $d_{in} \times d_{out}$ at rank r , it trains $r \times (d_{in} + d_{out})$ parameters.

For one 4096×4096 attention matrix, full fine-tuning updates 16.8 million parameters; LoRA at rank 8 trains 65,536, about 0.4 %. We applied the same formula to the configuration in our training script: rank 16 on all seven projection matrices of every layer.

Base model	Layers	Hidden / MLP / key-value width	Trainable parameters at $r = 16$	Share of the model
Qwen2.5-7B-Instruct	28	3,584 / 18,944 / 512	40.4 million	0.53 % of 7.61 billion
Qwen2.5-1.5B-Instruct	28	1,536 / 8,960 / 256	18.5 million	1.2 % of 1.54 billion

Table 4. LoRA size for our training configuration, computed from each model's published architecture.

Forty million parameters train in an hour or two on one consumer NVIDIA GPU. A full fine-tune of the same 7B model would need gradients and optimiser state for all 7.6 billion weights. With AdamW in mixed precision that is roughly 16 bytes per parameter, so about 120 GB of GPU memory. That was never an option for us.

3.4 What we decided

We built on three things.

1. **Structured prompting.** One system prompt per task: chat, cooking mode, recipe writing, recipe import, receipt reading. Each states the role, the rules of the kitchen and the expected output format.
2. **RAG.** Before writing a recipe, the server searches the corpus for similar recipes, or for the real recipe of a dish the user named. The user's stock is sent with every request.
3. **Strict checking.** Everything the model returns is treated like untrusted user input and checked before the app uses it (section 5.4).

LoRA stays as a second stage. The scripts exist and would mainly teach the model our recipe format and ingredient ids. Section 7 explains why we did not run them this week.

4. Hardware and model choice

4.1 The machines

Our demo machine is a **MacBook Air with an Apple M4 chip (4 performance and 6 efficiency cores) and 16 GB of unified memory**. "Unified" matters: there is no separate graphics memory. The operating system, the browser, the language model, its working cache and both voice models all share the same 16 GB. Some team members use Windows PCs, with or without an NVIDIA GPU, where models run as GGUF files through llama.cpp.

Before choosing a model, we sketched a memory budget. These are estimates from file sizes, not profiler measurements.

What	Approximate memory
Gemma 4 E2B, 4-bit weights	3.6 GB (4.1 GB on disk with tokenizer files)
Working cache for an 8,000-token context	0.5 to 1 GB
Parakeet 0.6B, speech recognition	about 1.2 GB
Kokoro 82M, speech synthesis	about 0.3 GB
Total for AI	about 6 to 7 GB, leaving the rest for macOS and Chrome

Table 5. Rough memory budget on the demo Mac.

A 7B model in 4-bit weighs close to 5 GB before its cache. On paper it fits. In practice it leaves the voice models and the browser fighting for what remains, and everything slows down.

4.2 How we ended up with Gemma 4 E2B

We did not pick the model from a leaderboard. We changed our mind several times, and each change came from something we saw on the machine.

1. **Qwen2.5-7B-Instruct was the plan.** It is available in every format, handles 29 languages including French, writes reliable JSON, and is what our training scripts target. On the Mac it was too heavy to sit next to the voice models.
2. **Qwen 3.5 9B looked like an upgrade.** It is a hybrid reasoning model. With its thinking mode left on, it spent 26 minutes reasoning before writing one recipe. With thinking off it worked, but at about 7.5 GB it was even heavier.
3. **Smaller models were fast but weak.** Llama 3.2 1B and Qwen2.5 3B answered several times faster and wrote noticeably worse recipes. A 4B Qwen 3.5 once asked for 380 g of salt for three people.
4. **Gemma 4 E2B was the balance point.** About 3.6 GB, about 15 tokens per second on the Air, better recipes than the 4B Qwen in our tests, and room left for voice.

To stop guessing, we wrote a small script, `check_model.py`. It asks the model server which model is really loaded, times a short answer from cold, checks whether the model reasons before answering, and times a real recipe to see whether it finishes. Most of the numbers in this section come from it.

4.3 Gemma 4 E2B in numbers

Property	Value	Source
Parameters	2.3 billion "effective", 5.1 billion including embeddings	Model card
Layers and attention	35 layers; most attend over a 512-token sliding window, some attend globally	Model card
Context length	128K tokens; LM Studio loaded it at 64,768 on our Mac; we size prompts for 8,192	Card, measured, our config
Languages	Trained on 140+, 35+ supported out of the box, French included	Model card
Benchmarks (instruction-tuned)	MMLU Pro 60.0, multilingual MMLU 67.4, GPQA Diamond 43.4, LiveCodeBench v6 44.0	Model card
Licence	Apache 2.0	Model card
Our build	4-bit MLX conversion, 3.55 GB of weights	Hugging Face repository
Speed on our Mac	About 15 tokens per second	Measured

Table 6. Gemma 4 E2B.

Two design choices explain why such a small model does well enough for us. **Per-layer embeddings** put a large share of the parameters into lookup tables that are cheap to store and cheap to use. The model has 5.1 billion parameters, but at inference it costs roughly what a 2.3 billion model costs. **Sliding-window attention** means most layers only look at the last 512 tokens, so the working cache grows slowly with context. On a 16 GB machine, both matter.

4.4 Sampling settings

Google recommends a temperature of 1.0 for Gemma 4. That is advice for open conversation. Almost everything we ask for is constrained: JSON with ingredient ids that must exist, quantities a person can eat, steps that must not mention food the recipe does not contain. We run recipe writing at temperature 0.2, with one retry at 0 if the JSON does not parse, and chat at 0.3. The chef sounds a little plainer. It also makes far fewer mistakes, which matched course section 7 on decoding.

4.5 Voice

Job	What we use	Alternative we considered	Why
Hearing (speech to text)	Parakeet TDT 0.6B on the Mac GPU; Whisper-base on CPU elsewhere	Whisper-small	Whisper-small makes about half the errors of Whisper-base but is 3× larger and about 2× slower. On a CPU, speed won. On the Mac, Parakeet is faster than both.
Speaking (text to speech)	Kokoro-82M	Piper	Piper is lighter and faster but sounds robotic. For something that talks to you for twenty minutes, voice quality won. A short phrase takes 0.15 to 0.25 s to generate on the Mac.
Wake word	The browser's speech recogniser listening for "Bob"	openWakeWord on device	The browser option worked in a day. Its downside: Chrome sends that audio to Google, so it needs internet.

Table 7. Voice components.

If Kokoro fails to load, the app falls back to Piper and then to the operating system's voice, so the assistant never goes silent. French has only one Kokoro voice, which is the main weakness here.

4.6 Data

Need	Chosen	Rejected	Why
Recipes	Kaiser1308/CookingRecipes: title, ingredients, directions, source link and ingredient names in separate columns; 2,119,522 recipes indexed	Khushis04/recipe-suggester-data; corbt/all-recipes	The largest structured set, and it keeps the source of every recipe. The first alternative has no source. The second stores each recipe as one block of text, which breaks retrieval when ingredients get separated from their steps.
Products on receipts	Open Food Facts, products sold in France: 483,210 products, 329,783 of them linked to one of our ingredients	Letting the model read receipts alone	A till line like "CRF LT DEMI ECR 1L" is an abbreviation puzzle. Searching real product names solves it better than a small model guessing.
Ingredient vocabulary	Our own catalogue: 3,052 ingredients with name, category and unit	Free-text ingredient names	Ids are what make checking possible. An invented id can be caught and dropped.
Substitutions and nutrition (prototype)	oraclemangle/historical-culinary-substitutions; omid5/usda-fdc-foods-cleaned	lishuyang/recipepairs; the full Open Food Facts nutrition export	Ingredient-level swap rules and a lighter nutrition table. Indexed in our ChromaDB prototype, not yet in the app.

Table 8. Datasets.

One honest caveat: the recipe corpus is mostly American home cooking in English. Its most typical carbonara uses bacon and butter. The assistant follows what the corpus says a dish is, and we would rather say so than pretend otherwise.

5. How the application works

5.1 What the user sees

The app has five screens: Today (tonight's suggestion), Kitchen (stock and expiry), Recipes (saved recipes), Receipt and Shopping. On top of them sit the chef chat and cooking mode. A typical evening looks like this:

1. You come home and photograph the receipt, or say "Bob, I bought 300 grams of chicken". The stock updates with an expiry estimate.
2. You say "Bob, I'm drained, give me 15 minutes". The app writes one dinner from what is already in the kitchen, with nothing to buy, and opens it.
3. You press Start cooking. The chef reads one step at a time, tells you what to look and listen for, starts timers, and answers questions about the step. You say "Bob, next" to move on.

5.2 Architecture

We split the app into two layers. **CORE** is everything that does not need a model: stock, expiry, shopping list, recipe book, cooking history. It runs entirely in the browser and stores data locally, so it still works when the server is off. **EXTRA** is the AI layer: chat, recipe writing, cooking mode, receipt reading, voice. Each EXTRA feature switches itself off and says why if its model or package is missing.

The server keeps no household data. The browser sends the stock with each request, and nothing is stored server-side. The model can change the kitchen only through eight tool calls (add stock, adjust stock, set expiry, add an ingredient, save a recipe, add to shopping, set a timer, open a screen), and each call is checked against the catalogue. In short, the model can only do what you could already do by tapping.

```
voice or text
|
v
Parakeet / Whisper (speech to text)
|
+--> stock and profile (from the browser)
+--> ingredient ids (catalogue, trimmed to fit)
+--> similar or reference recipes (SQLite full-text search)
|
v
Gemma 4 E2B in LM Studio (OpenAI-compatible API, on the laptop)
|
+--> recipe JSON --> clean and check --> retry once if wrong
+--> chat text, streamed --> sentence by sentence --> Kokoro (voice)
+--> tool call --> checked --> applied in the browser
```

Figure 1. What happens between a question and an answer.

5.3 Fitting everything into the context window

This is where local models break most often, so we handle it explicitly.

The number that matters is not the 128K on the model card but the window the model was actually loaded with, and how long the user is willing to wait. On a laptop, every token of prompt is read before the first word of the answer. The server asks LM Studio or Ollama what window the model was loaded with, caps prompts at 8,192 tokens, and keeps 35 % of that for the answer.

The most expensive part of a prompt is the ingredient list. Sending all 3,052 ids would be about 34,000 tokens. The server keeps what is in the kitchen first, then seasonings, then as much of the rest as fits. Anything that does not fit can still appear in a recipe as plain text. We learned the limits by breaking them. With a 4,000-character ingredient list in the recipe prompt, Gemma started returning recipes with no ingredients at all, so chat and recipe writing now have separate budgets.

Finally, recipe requests send the JSON schema to the model server, which then only allows tokens that keep the JSON valid. Malformed JSON went from occasional to impossible.

5.4 Never trusting the model's output

Everything the model returns goes through checks written in plain Python before the app uses it:

- **Cleaning.** Invented ingredient ids are dropped. Impossible quantities are brought down to normal portions, and the recipe says so. Seasonings are sorted according to the catalogue, not the model's opinion.
- **Problem detection.** Do the step times add up to the time the person has? Does a step mention a food that is not in the recipe? Is a sauce used before the step that makes it? Does the oven get turned on before something is baked?
- **One retry.** If problems are found, the model gets the exact list and one more try. If the second attempt is still wrong, the better one is shown with a "check this before you start" box.
- **Named dishes.** When the user asks for a specific dish, the result is compared with the real corpus recipe it was based on, and sent back if a key ingredient or cooking method disappeared.

These checks came from real failures: 380 g of salt for three people, 1.7 kg of courgettes, a litre and a half of olive oil, a step lasting 255 minutes.

5.5 Reading a receipt

A receipt goes through three stages. **Tesseract**, an open-source OCR engine running locally, turns the photo into lines of text. Each line is then **looked up** in the Open Food Facts index: abbreviations are expanded, the database is searched by word fragments so "ECR" can find "écrémé", and candidates are scored word by word. Finally **Gemma** reads each line together with what the lookup found and decides the ingredient and quantity. When the model and the database agree, confidence goes up. When they disagree, the line is marked "Check" for the user. Corrections the user makes are remembered for the next receipt.

Getting a usable photo was its own small project. The first version used a file input, which on a laptop opens a file picker, not the camera. We replaced it with a live camera view. That view looked zoomed in and blurry, and measuring the camera showed why: we had asked for a 4K widescreen frame the MacBook camera does not have, so Chrome gave us a narrow 1080 × 1920 slice of the image. Asking the camera for its own largest frame gives 1552 × 1552 with the full field of view. We also added a review step so a blurry photo can be retaken before a minute is spent reading it.

6. Retrieval in practice

6.1 Why full-text search and not embeddings

The textbook RAG in course section 11.2 uses embeddings in a vector database. Our Data Engineer built exactly that as a prototype: bge-small-en-v1.5 embeddings stored in ChromaDB, with separate collections for recipes, substitutions and nutrition. It works. We still shipped something simpler.

Criterion	Embeddings in ChromaDB (prototype)	SQLite FTS5 with BM25 (shipped)
Indexing 2.1 million recipes	One model pass per recipe: hours on a laptop, about 15 GB of disk. The prototype stops at 50,000 recipes per collection.	Built in a background thread while the app already runs; 2.9 GB index; rebuilt only when the corpus changes.
Extra dependencies	PyTorch, sentence-transformers, ChromaDB	None beyond Python itself
Understands meaning	Yes: "something cosy with chicken" finds stews	No, it matches words. Helped by stopwords, accent folding and dish detection.
Query time	A few milliseconds on 50,000 rows	3.3 s at first; tens of milliseconds after the fix in 6.2

Table 9. The two retrieval engines, measured on the demo Mac.

The size of the corpus against the size of the laptop decided it. Our queries are mostly lists of ingredients plus perhaps a dish name, which word matching handles well. Semantic re-ranking of the best word matches is our planned next step.

6.2 From 3.3 seconds to a few milliseconds

Our first version was slow for an instructive reason. The query was a sentence plus the whole stock list, so it always contained words like "salt", which appears in about half of all recipes. The ranking algorithm had to score a million recipes on every request: 3.3 seconds before the model even started.

The fix: drop words that cannot tell recipes apart (grammar, units, anything in more than a quarter of the corpus), keep the five rarest words, and require all of them. If too few recipes match, drop the most common of the five and try again. Search now takes tens of milliseconds, and the results are better: a recipe with chicken and rice and black beans, rather than whichever recipe says "chicken" most often.

6.3 Named dishes and "template first"

Early on, "make mochi" returned a bruschetta. The ingredient code "oliveoil" in the stock list happened to be rarer in the corpus than the word "mochi". So the server now recognises when a request names a dish. A word counts as a dish when the corpus uses it in recipe titles at least as often as in ingredient lists: "carbonara" 55 times more, "chicken" much less. For a named dish, the server picks the most typical version, the one whose ingredients most other versions share, and asks the model to follow it.

For an ordinary dinner, the server first looks for a corpus recipe the kitchen can already make: at least 72 % of its ingredients in stock. If it finds one, the model only converts that recipe into our format instead of inventing a dish. Converting is easier and more reliable for a small model. There is an exception, which we found by measuring: for a 15-minute dinner, converting a corpus recipe took 48 seconds and then failed the time check, while writing a short recipe from scratch took 20 to 25 seconds. Quick dinners skip this step.

7. Fine-tuning: built, not run

The training folder contains a complete LoRA pipeline using Hugging Face transformers, peft, trl and datasets:

1. **build_dataset.py** streams the recipe corpus without loading it into memory and writes chat-style training examples, split into training and validation sets.
2. **build_app_dataset.py** does the same in the app's own recipe format and reports how many source ingredients map to a catalogue id. That figure is also a data-quality measure.
3. **train_qlora.py** trains LoRA at rank 16 (alpha 32, dropout 0.05) on all attention and MLP projections: two epochs, effective batch size 16, learning rate 2e-4. On NVIDIA it loads the 7B model in 4-bit (QLoRA). On the Mac it trains the 1.5B model without quantisation, because the 4-bit library only supports NVIDIA.
4. **test_model.py** runs held-out prompts through the tuned model and through the untuned baseline, and reports the share of answers that are valid recipes.
5. **export_ollama.py** merges the adapter into the model so it can be served through the same API as any other model.

Why we did not run it. Our only capable GPU was also the demo machine. Schema-constrained decoding and the recipe checks had already removed most of the failures a fine-tune would target. Each training run would take hours we preferred to spend on the parts the user touches. The README says plainly that the pipeline has self-tests but has not been run end to end.

There is also a mismatch we should name. The scripts target Qwen2.5, our original choice, while the final model is Gemma 4 E2B. Pointing them at Gemma means changing the base model and checking its chat template. That would be the first task of a second stage, followed by measuring the valid-recipe rate against the untuned baseline.

8. Integration and deployment

Layer	Technology
Frontend	React 18 and Vite; data stored in the browser (IndexedDB); unit tests for the non-AI logic
Backend	Python 3.11/3.12, FastAPI, streamed answers (server-sent events)
Language model	LM Studio (MLX on Mac) or Ollama (GGUF), both exposing an OpenAI-compatible API; a cloud provider is possible but not used in the demo
Voice	Parakeet or Whisper for hearing, Kokoro for speaking; weights downloaded from Hugging Face on first use
Retrieval	SQLite FTS5
Receipts	Tesseract OCR (French and English), Open Food Facts SQLite index, the language model
Running it	Three processes during development (model server, backend, frontend); one address once the frontend is built

Table 10. Technology stack.

A few integration decisions saved us repeatedly:

- **One API for every model.** The backend only speaks the OpenAI chat format to a local server. Switching between Llama 3.2 1B, Qwen2.5 3B, Qwen 3.5 and Gemma 4 E2B during the week meant editing one line of configuration.
- **Every feature optional.** A missing package turns one feature off and the status endpoint explains why. Team members with different machines could all run the same code.
- **Tools that measure.** `check_model.py` measures the model server. `audit.py` reads a recipe book like a cook would and lists mistakes. `check_voice.py` tests the voice chain end to end.
- **Privacy by default.** Everything runs locally. The server stores no household data, profiles contain no health data, and the recipe importer refuses private network addresses.

9. Testing and evaluation

9.1 Why not perplexity

Perplexity measures how well a model predicts a reference text. There is no reference text for "dinner from this fridge". A fluent recipe that uses hummus you don't have is a failure that perplexity cannot see. Course section 12 makes the same point about BLEU and ROUGE, so we measured things tied to actual use.

9.2 What we measured

What	How	Result
Speed of the model	<code>check_model.py</code> against LM Studio	About 15 tokens/s for Gemma 4 E2B on the Air; a full recipe of about 700 tokens takes roughly 45–50 s (estimated from that rate)
Quick dinner	Timed on the demo Mac with a test fridge	20–25 s for a 15-minute recipe
Retrieval time	Timed per request	3.3 s before the query fix, tens of milliseconds after
Valid JSON	Server parse step	Malformed JSON no longer occurs with schema-constrained decoding
Hallucinated ingredients and portions	Recipe checks on every request	Caught and sent back once; remaining problems shown to the user, not hidden
Voice	Server logs	Voice models warm in about 9 s at start-up; a short phrase takes 0.15–0.25 s to synthesise
Receipt camera	Camera settings read in Chrome	1080 × 1920 cropped frame before, 1552 × 1552 full view after
Receipt data	Database counts	483,210 products; 329,783 linked to a catalogue ingredient

Table 11. What we measured.

9.3 An experiment on the cooking assistant

While testing a rice recipe, we noticed the chef giving odd answers. Asked "I want to eat rice" on step 2 ("add the rinsed rice and tomato to the pot"), it answered from step 3: "keep simmering until the liquid is absorbed". We replayed the same three questions through the real prompt, three times each, changing one thing at a time.

Setup	Answer stayed on the current step
As the app sends it, with conversation history	0 of 3 runs
Same, with the recipe corrected	0 of 3 runs
No conversation history	3 of 3 runs
History, with the current step repeated inside the question	3 of 3 runs
No history, with the current step repeated inside the question	3 of 3 runs

Table 12. Does the chef answer about the right step? Gemma 4 E2B, temperature 0.3.

The cause is not the recipe. In our prompt, the current step appears once, at the end of the system prompt, and the conversation history follows it. A 2-billion-parameter model follows the conversation rather than a line far above it. Repeating the step inside each question fixed it in every run. We have not shipped that fix yet; it is the first item in section 10.3. We like this experiment because it is exactly what the course means by evaluating on real usage: a small, controlled test that turned a vague "the chef is confused" into a cause and a fix.

9.4 Incidents that changed the design

- **The 26-minute recipe.** A reasoning model thought for 26 minutes before answering. We now turn thinking off in every request, strip any reasoning text before parsing, and `check_model.py` reports whether reasoning is on.
- **The 437-second wait.** The HTTP client retried twice silently, so a four-minute failure looked like a twelve-minute hang. We now make one attempt with a long timeout and explain what to change when it expires.
- **34,000 tokens into an 8,192-token window.** We sent the whole ingredient catalogue. The server now asks what window the model has and budgets the prompt.
- **Prices read as quantities.** Gemma once added 2.31 courgettes and 7.45 cl of olive oil to the stock: those were the prices. A size printed on the line (1L, X6) now wins, and a quantity equal to the price is thrown out.

10. Limits and next steps

10.1 What does not work well yet

- **The chef can drift to the wrong step** after a few questions (section 9.3). The fix is known and tested but not shipped.
- **Recipe checks miss some errors.** A rice recipe with no water, whose step 3 simmers "until the liquid is absorbed", passed our checks. The checks catch wrong facts, not every cooking mistake.
- **Ingredient lists can be padded.** When the model lists the whole fridge as extras, each item becomes an ingredient with no quantity.
- **Retrieval matches words, not meaning,** and the corpus is mostly American home cooking.
- **French has one Kokoro voice.**
- **The wake word needs internet,** because Chrome's recogniser sends audio to Google while listening for "Bob".

- **A laptop camera is about 2.4 megapixels with fixed focus.** A phone photo imported into the app reads better. iPhone HEIC photos are not supported yet.
- **Fine-tuning is not run,** and the scripts still target Qwen2.5.
- **One household, one browser.** Data is not synchronised between devices.

10.2 Risks we thought about

Risk	Where it shows up	What we do about it
Hallucination	Invented ingredients, quantities or steps	Retrieval from real recipes, schema-constrained output, checks with one retry, problems shown to the user
Unsafe advice	380 g of salt	Per-person caps; above the cap the amount becomes "to taste"
Over-reliance	Trusting a wrong time or heat	The prompt favours visual cues over clock times; time and heat checks
Bias	The corpus decides what a dish "is"	Stated openly; the reference recipe is visible
Privacy	What is in someone's kitchen	Local by default, no server-side storage, no health data

Table 13. Risk register, following course section 15.

10.3 Next steps

1. Repeat the current step inside every cooking-mode question, and re-run the experiment in 9.3 to confirm.
2. Add a check for recipes that simmer or absorb liquid that was never added, and stop turning unused extras into ingredients.
3. Point the fine-tuning scripts at Gemma 4 E2B, run them on an NVIDIA machine, and compare the valid-recipe rate with the untuned model.
4. Re-rank the best full-text matches with embeddings, and bring substitutions and nutrition from the prototype into the app.
5. Replace the browser wake word with an on-device model, and add HEIC support for phone photos.

11. What we learned

Measure before you argue. Our worst hours were spent guessing why the model was slow or wrong. A small script that prints tokens per second and whether the model is thinking answered in a minute questions we had been guessing about for much longer.

With a small model, most of the work is around the model. Choosing the model was a small part of the week. Making it dependable was most of it: budgeting the prompt, constraining the output, and checking every answer. The course material on prompting, decoding and RAG turned out to be more useful day to day than the material on training.

Deterministic beats clever where it can. A check that says "these steps add up to 45 minutes and you have 20" is exact. A fine-tune that makes such mistakes less likely is statistical. We used the model for what only a model can do, and plain code for the rest.

The unglamorous parts decide whether a demo works. Repositories, folder structures, Python versions, a camera returning a cropped frame: none of it is AI, and all of it could have sunk the project. Next time we would name one person responsible for the environment from day one.

12. Conclusion

KitchenBuddy shows what running a generative model on student hardware really involves. The constraint pushed us toward prompting, retrieval and strict checking, rather than a fine-tune that looks attractive on paper but fits neither our compute nor knowledge that changes every day. We can defend that choice with numbers: the knowledge lives in a fridge and a corpus of 2.1 million recipes, not in weights, and a fine-tune would cost 40 million trainable parameters and hours of GPU for problems our checks already catch.

The final system runs Gemma 4 E2B on a MacBook Air next to local speech recognition and speech synthesis. It reads receipts, writes a quick dinner in under half a minute, and talks a person through cooking it. It still makes mistakes, and we know several of them precisely because we took the time to measure. We think that is the most useful thing this project taught us.

13. References

Models

- Gemma 4 E2B-it (Google): <https://huggingface.co/google/gemma-4-E2B-it> ; 4-bit MLX build: <https://huggingface.co/mlx-community/gemma-4-e2b-it-4bit>
- Qwen2.5-7B-Instruct: <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct> ; Qwen2.5-1.5B-Instruct: <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>
- Qwen 3.5 9B: <https://huggingface.co/unsloth/Qwen3.5-9B-GGUF>
- Parakeet TDT 0.6B v3 (MLX): <https://huggingface.co/mlx-community/parakeet-tdt-0.6b-v3>
- Whisper-base and Whisper-small (OpenAI): <https://huggingface.co/openai/whisper-base>
- Kokoro-82M: <https://huggingface.co/hexgrad/Kokoro-82M> ; Piper: <https://github.com/rhasspy/piper>
- bge-small-en-v1.5: <https://huggingface.co/BAAI/bge-small-en-v1.5>

Datasets

- Kaiser1308/CookingRecipes: <https://huggingface.co/datasets/Kaiser1308/CookingRecipes>
- Open Food Facts: <https://world.openfoodfacts.org>
- oraclemangle/historical-culinary-substitutions: <https://huggingface.co/datasets/oraclemangle/historical-culinary-substitutions>
- omid5/usda-fdc-foods-cleaned: <https://huggingface.co/datasets/omid5/usda-fdc-foods-cleaned>
- Alternatives considered: Khushis04/recipe-suggester-data, corbt/all-recipes, lishuyang/recipepairs

Papers and course

- Hu et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models.

- Dettmers et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs.
- Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
- Radford et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision (Whisper).
- Robertson and Zaragoza (2009). The Probabilistic Relevance Framework: BM25 and Beyond.
- Generative Artificial Intelligence course notes, sections 6 to 12, 15 and 18.

Tools

- LM Studio, Ollama, llama.cpp, MLX; Hugging Face transformers, peft, trl, datasets; FastAPI; React and Vite; SQLite FTS5; ChromaDB; Tesseract OCR.